Swiss Institute of Bioinformatics

Dalle Molle Institute for Artificial Intelligence USI–SUPSI

Federated Learning in Bioinformatics

SIB Training Course

Practical Sessions (Handout)

Instructors

Dr. Daniele Malpetti, *Lecturer–Researcher*
Dr. Francesco Gualdi, *Researcher*
Dr. Laura Azzimonti, *Senior Lecturer–Researcher*

Dalle Molle Institute for Artificial Intelligence (IDSIA), SUPSI

SIB Group: Machine Learning for Bioinformatics and Personalised Medicine

27 April 2026 — Lugano

`https://github.com/sib-swiss/federated-learning-training`

Overview

These practicals emulate, in simplified form, a realistic collaborative workflow. Seven institutions worldwide each hold pancreas single-cell datasets, with each dataset generated using a different sequencing technology. They collaborate to train an open-source model for technology-related batch-effect removal, using an architecture provided by **scVI-tools**, and plan to disseminate it through a journal publication. Since data cannot be shared across sites, the partners adopt **Federated Learning (FL)** with the **Flower** framework, allowing training to occur locally while only model updates are exchanged.
A separate institution, which also has pancreas single-cell data, later reads the publication and wishes to use the released model. This is particularly useful for them because their study contains measurements from two different technologies (one machine failed mid-study and was replaced), and technology-related batch effects are present. They therefore need to remove these batch effects to perform several downstream tasks.

Plan for the practical

[Morning] Exercise 1. We will perform a minimal FL workflow to become familiar with Flower and the mechanics of a federated process. Concretely, we will identify *highly variable genes*. We will show that running this computation on centralized data and running it in a federated manner produces the same result.

[Afternoon] Exercise 2. We will train a deep-learning scVI model in a federated setting, and also train the same model in a centralized manner. Finally, we will compare the effectiveness of the federated and centralized approaches on data from a new (previously unseen) institution.

[Afternoon] Exercise 3. We will conduct a live federated training session in small groups using the code you have developed: one participant will act as the server, and the others as clients.

Exercise 1 — Federated highly variable genes: a preprocessing step

Goal

We will familiarize ourselves with Flower and the mechanics of a federated workflow by implementing a minimal end-to-end project. Instead of training a deep model, we will identify the top-k most highly variable genes in a federated dataset.

Steps

1. **Pull from Git.** In VS Code, since you have already cloned the course repository, you only need to pull the latest updates. Open the **Source Control** panel from the left sidebar. Next to **CHANGES**, click the **...** menu, then select `Pull`.

2. **Download data.**

   Create a folder named **data_centralized** and download the dataset from the following link into that folder:

   `https://drive.switch.ch/index.php/s/J8NLl6KBVWwQEEA`

3. **Preliminaries.** Activate the conda environment `fl-course-env` with:

   ```
   conda activate fl-course-env
   ```

   Then run:

   ```
   python preliminaries.py
   ```

   This script downloads and prepares all datasets required for the course.

   Inspect the folders that are created:

   - **Centralized data folder:** contains the datasets as they will be used for model training: one for training and one for validation, together with two metadata files, namely the ordered list of all genes and the list of all sequencing technologies in the dataset. This folder also contains the external test set that will be used to compare the centralized and federated models.
   - **Federated data folder:** contains one folder per client and one folder for the server. The client folders contain the local portions of the training and validation data, together with the metadata files; the server folder contains only the metadata files, and no expression data.

   This reflects a realistic federated setting: clients keep their own data locally, while the server coordinates the process without direct access to the datasets.

4. **Centralized analysis.** In the `exercise-hvg` folder, run the `centralized_analysis.ipynb` notebook to familiarize yourself with the `AnnData` object type and inspect the printed outputs.

   Pay particular attention to the two helper functions that decompose the built-in HVG routine into separate steps. This provides an example of how a standard centralized

computation can be reformulated in an additive form, which is essential in our federated settings.

Observe how the same final result can be recovered without directly pooling the raw data.

5. **Inspect the main files of the Flower app.** Review the structure of the `exercise-hvg` folder. At the moment, most of the final code is still commented out. The first version of the app simply generates random numbers on each client, sends them to the server, and computes their average. Only the following files are involved:

   - `pyproject.toml`: configures the Flower app. Have a look at the parameters, even though most of them are not used yet.
   - `app/client_app.py`: defines the client-side Flower application.
     - Creates a `ClientApp`.
     - In the active toy example, each client sends one random local value to the server during training, together with an associated weight of one.

   In this minimal example, no real model is trained. Instead, clients generate random integers, and the server aggregates them using FedAvg with equal weights. This provides a simple working skeleton that you will gradually adapt to identification of highly variable genes.

6. **Run your first FL project.** In the terminal, execute:

   ```
   flwr run exercise-hvg
   ```

   Observe the log output, which is mostly minimal since the process is still trivial.

7. **Include evaluation.** We now introduce communication from the server back to the clients. This is done through a first trivial evaluation endpoint defined in the client app file. Uncomment that block. As a shortcut to uncomment blocks of code in VS Code, you can use `Ctrl + /` on Windows/Linux or `Cmd + /` on Mac. In the server file, set the evaluation fraction to `1.0`, so that the evaluation step is executed by all clients.

   In the evaluation, each client simply receives the average of the random integers computed by the server and prints it locally. Since we are running in simulation mode, all client outputs will appear in the same terminal. In a real federated deployment, each client would print the result in its own local terminal.

8. **Calculate local statistics.** As we saw in the centralized execution, the main quantities needed to compute the HVGs with the helper functions are:

   - the gene-wise sums of expression values,
   - the gene-wise sums of squared expression values,
   - the total number of cells.

Delete or comment out the trivial `train` function currently in use, and uncomment the actual training function provided below it.

Inspect the function: it reads the local client data, computes the required local statistics using the custom helper function (imported from `task.py`), and sends them to the server.

At this point, the client-side training step is ready. We now still need to modify the server-side aggregation, since these statistics must be summed correctly, and then decide what should be sent back to the clients.

9. **Custom aggregation strategy.** We can now modify the server-side behavior, since at the moment it only computes the average of a multi-dimensional array.

   We do this through a custom strategy, defined in the `custom_strategy.py` file. The strategy inherits from `FedAvg`, but extends its behavior with additional logic.

   In particular, it uses the aggregated client outputs to recover the global sums (rather than averages), since the built-in FedAvg returns weighted averages by default. It then applies our helper function to compute the global statistics needed to identify the highly variable genes, and also saves the list of highly variable genes on the server side.

   At this stage, the server cannot simply send back the gene names, because Flower communication between server and clients is based on messages containing NumPy arrays.

   For this reason, the server instead creates a binary mask of length equal to the number of genes:

   - value `1`: the gene is selected as highly variable,
   - value `0`: the gene is not selected.

   Since both the server and all clients share the same ordered gene list, each client can use this mask locally to recover the corresponding gene names.

10. **Include evaluation.** You can now comment out the trivial evaluation block and uncomment the actual evaluation block provided below.

    This evaluation step receives the mask sent by the server, uses the ordered gene list to recover the selected gene IDs, and saves them locally.

    When you run the project again, you should see a new output file appear inside each client folder.

11. **Compare centralized and federated results.** Now that you have both the centralized and federated versions of the highly variable gene list, open and run the notebook `results_comparison.ipynb`.

    Verify that the two outputs are identical.

    The notebook compares the result for client 0, but the same output should be obtained for every client, since the information received from the server is identical across all clients.

Exercise 2 — Federated scVI: training, monitoring, and model release

Goal

We will train an scVI model in a federated simulation using Flower, save the final aggregated model, monitor training and validation losses across rounds, add early stopping, and compare the federated model against a centralized baseline.

Steps

1. **Run the centralized baseline.** In the `exercise-scvi` folder, open `centralized_training.ipynb` and run it. This notebook trains an scVI model on the centralized training set, and saves the resulting model. Skim the code to become familiar with the main scVI commands and conventions.

2. **Inspect and run the federated project.** Navigate to `exercise-scvi` and launch the Flower simulation:

   ```
   flwr run exercise-scvi
   ```

   While the simulation runs, inspect the code structure in `exercise-scvi/app`:

   - `server_app.py`: defines the server-side workflow. Since the server has no real data, it creates a *dummy* `AnnData` object using the highly variable genes and batch categories. This dummy object is used only to initialize the scVI model architecture and registry. The initial model weights are then passed to Flower as the starting point for federated training.

   - `client_app.py`: defines the client-side training and evaluation logic. Each client loads its local training and validation data, restricts them to the highly variable genes, initializes an scVI model, receives the current global weights, trains locally, and sends updated weights back to the server.

   - `custom_strategy.py`: contains several custom strategies of increasing complexity. These strategies extend Flower's `FedAvg` to save the final model, log losses, plot training curves, and implement federated early stopping.

   - `task.py`: contains reusable helper functions used by both server and clients, including data loading, creation of dummy and local `AnnData` objects, scVI model initialization, extraction and loading of model weights, loss computation, configuration parsing, and saving of the final trained model.

   At this stage, the simulation runs end-to-end, but only the basic federated training behaviour is enabled.

3. **Save the final federated model.** In `server_app.py`, switch from the plain `FedAvg` strategy to `FedAvgSaveModel` and re-run the training. This strategy saves the final aggregated server-side model at the end of training.

   Verify that a saved model appears in the configured output folder (`model_federated`). In this practical, we save one global model on the server side. This mimics the situation where the consortium releases a single trained model for external use.

4. **Save the model and monitor losses.** In `server_app.py`, switch from the plain `FedAvg` strategy to `FedAvgSaveModelPlotLosses`. This strategy:

   - saves the final aggregated federated model on the server side,
   - collects `train_loss` and `valid_loss` from each client,
   - computes global weighted averages of both losses,
   - updates a loss plot after each round.

   Re-run the training and verify that a saved model appears in the configured output folder, for example `model_federated`, and inspect the generated loss plot.

5. **Experiment with local epochs and federated rounds.** Keep the total amount of local training approximately fixed at 50 epochs, but vary how it is split between local training and federated communication rounds in `pyproject.toml`. For example, try:

   - 1 local epoch and 50 federated rounds,
   - 5 local epochs and 10 federated rounds,
   - 10 local epochs and 5 federated rounds.

   This can be done in groups, with each person in the group testing a different configuration. Compare the results. What trends do you observe? In terms of computational time? In terms of the loss behaviour?

   Based on your observations, if you want, you can update the value of `num_local_epochs` in `pyproject.toml`.

6. **Enable early stopping.** Set `num_rounds` in `pyproject.toml` to a large value, for example 100. Then switch to `FedAvgSaveModelPlotLossesEarlyStopping` in `server_app.py`.

   This strategy monitors the global weighted validation loss. If the validation loss does not improve sufficiently for a specified number of rounds, training stops early and the best model is saved.

   The value chosen in the toml for the delta of early stopping is larger than what most likely you would use in practice, but this is meant to keep the training time reasonable with respect to the duration of the course.

   Re-run the simulation and check that:

   - the loss plot stops before the maximum number of rounds,
   - the console prints an early-stopping message.

7. **Compare centralized and federated models.** Open `model_comparison.ipynb` and run it. This loads both the centralized and federated models, computes their latent representations, and visualizes the resulting UMAPs.

   You should observe that both models reduce technology-related batch effects. In this practical, we focus on visual inspection. A rigorous comparison would require quantitative metrics and repeated runs, which are beyond the scope of this session.

Exercise 3 — Live federated run

Goal

We will run a live multi-client federated training session: one participant will act as the server, and the others as clients. For this exercise, everyone will connect to the same Wi-Fi network provided through a local router. In a real federated setting, server and clients would typically operate across different networks, but that would require a more complex setup.

Even in this simplified setting, the process remains truly federated: the data stay on different laptops, while only model updates are exchanged. For details on production deployments, please refer to the Flower documentation.

Common steps

1. **Form groups.** Split into three groups of comparable size.
2. **Assign roles.** In each group, decide who will be the *server*; the other participants will be *clients*.
3. **Assign client IDs.** The server assigns each client a unique integer ID, `YOUR_ID`, ranging from 0 to `NUM_CLIENTS - 1`, where `NUM_CLIENTS` is the number of clients in the group.
4. **Activate the environment.** In your terminal, activate the course environment if it is not already active.
5. **Connect to Wi-Fi.**

   ```
   Network: Exercise-wifi
   Password: federated
   ```

6. **Network access (macOS only).** Enable network access for VS Code at:

   ```
   System Settings > Privacy & Security > Local Network
   ```

Server steps

a) **Adjust configuration files.** For both apps, check `pyproject.toml`. Since clients will run inside their own data folders, you can set:

```
client_folder = "."
```

b) **Get your IP address.** From the VS Code terminal, use the command appropriate for your operating system:

```
macOS:   ipconfig getifaddr en0
Linux:   hostname -I
Windows: ipconfig
```

Use the local network IP address corresponding to the Wi-Fi network as the `SUPERLINK_IP`.

c) **Start Flower SuperLink.** In your terminal, run:

```
$ flower-superlink --insecure
```

The flag `-insecure` disables encryption. This is acceptable for the exercise, but not for production use.

d) **Tell clients the IP address.** Share `SUPERLINK_IP` with the clients and ask them to connect. Keep your screen visible so everyone can see when clients join.

e) **Run preprocessing and training.** Open another terminal and activate the course environment there as well.

First run preprocessing:

```
$ flwr run exercise-hvg local-deployment --stream
```

Then run training:

```
$ flwr run exercise-scvi local-deployment --stream
```

Open the loss plot and observe it updating during training.

Client steps

a) **Move to your local data folder.** Go to the folder corresponding to your ID.

Example for client 0:

```
$ cd data_federated/data_client_0
```

b) **Remove previous HVG file.** This file will be generated again through the federated preprocessing step.

```
$ rm top2k_genes.json
```

c) **Compute your port.**

```
YOUR_PORT = 9094 + YOUR_ID
```

Example: client 0 uses 9094, client 1 uses 9095.

d) **Run the client.**

Make sure the course environment is active.

The server participant will provide the `SUPERLINK_IP`. Run:

```
$ flower-supernode \
    --insecure \
    --superlink <SUPERLINK_IP>:9092 \
    --clientappio-api-address 127.0.0.1:<YOUR_PORT> \
    --node-config "partition-id=<YOUR_ID>"
```